Apollo3 Architecture Refactoring Report

2026-03-16

Contents

Apollo3 Architecture Refactoring Report

What We Delivered

Apollo3 is a collaborative genome annotation editor built on JBrowse 2. This document describes architectural changes that expanded where and how Apollo3 can be deployed.

Database migration (MongoDB -> MikroORM). Previously, an entire gene was packed into a single MongoDB document. Every edit rewrote the whole gene, and two annotators editing different exons could silently overwrite each other. Now each feature is its own database row -- edits touch

only what they change, and concurrent edits to different features no longer risk overwriting each other.

Simplified dev and production setup. MongoDB required multiple Docker containers and orchestration scripts even for local development. Now SQLite is the default: `pnpm install && pnpm start` is the complete setup. For production, PostgreSQL needs only one container. This can reduce hosting costs and operational burden.

Per-assembly storage and permissions. Previously, a single config document held every assembly and track with no access control. Now each is its own record, and the server generates config per request filtered by user permissions. Assemblies can be public or private with per-user roles, enabling multi-team deployments on shared infrastructure.

Multi-page application. Previously, all of Apollo3's UI -- admin panels, organism management, user management -- was packed into the JBrowse plugin itself. This made the plugin heavy and made it difficult to add new pages or workflows. We moved to a multi-page Vite application where JBrowse is one page among several. This allows lightweight, purpose-built pages: a BLAST search portal, an assembly management dashboard, organism and user admin pages, and room for future additions like a recent changes feed.

Analysis tool integration. Added built-in BLAST sequence search (local and NCBI remote) and Tiberius deep learning gene prediction, both server-side. Annotators search for homology and generate predictions without leaving Apollo3.

Performance. 7-20x speedups across import, query, export, and delete. The largest win (20x): quality checks were re-running on every pan/zoom even when nothing changed. Checks now run only after edits.

Security. Fixed 7 pre-existing vulnerabilities including an open redirect for OAuth token theft, missing cookie security flags, and a WebSocket CORS wildcard.

Code simplification. Removed redundant abstraction layers, consolidated WebSocket channels, reduced developer setup from 4 parallel processes to 1, and eliminated legacy Mongoose schema packages.

Architecture Overview

Per-Assembly Storage

Previously, the server maintained a single JBrowse `config.json` document in the database that described every assembly, every evidence track, and every search adapter for the entire Apollo3 instance. When an admin added a track or modified an assembly, the server rewrote this entire document. More importantly, all users saw the same configuration -- there was no mechanism to show different assemblies or tracks to different users.

We replaced this with a normalized data model where each assembly, evidence track (BAM, VCF, BigWig, CRAM), BLAST database, and text search adapter is stored as its own database record. Tracks and BLAST databases are linked to assemblies through many-to-many relationships, so a single track can appear on multiple assemblies. The server now generates `config.json` dynamically for each request, including only the assemblies and tracks that the requesting user has permission to see.

This means teams can manage their own evidence tracks independently -- uploading, modifying, or removing tracks on their assemblies without admin intervention and without affecting other assemblies on the same instance.

Per-Assembly Permissions

Access control operates at two levels. Global roles (`none`, `readOnly`, `user`, `admin`) set a baseline for what a user can do across the instance. On top of this, admins can assign per-assembly roles that override the global default for specific assemblies. Each assembly also has a visibility setting -- public assemblies are visible as read-only to all authenticated users, while private assemblies are visible only to users with an explicit permission grant.

When a user accesses an assembly, the system resolves their effective role by checking (in order): global admin status, then per-assembly permission, then assembly visibility. This allows scenarios like a postdoc having edit access to their own genome, read-only access to a collaborator's public assembly, and no access to another lab's private data -- all on the same server.

Analysis Tool Integration

Apollo3 now includes server-side analysis tools that are linked to specific assemblies and tracked through the database.

BLAST sequence search allows annotators to search for homology against registered BLAST databases without leaving Apollo3. Admins register databases and associate them with assemblies, so annotators see only relevant databases. Jobs support all standard BLAST programs (blastn, blastp, blastx, tblastn, tblastx) and can run against either local databases or NCBI's remote servers. Job status and results are persisted in the database.

Tiberius gene prediction lets annotators select a genomic region and run the Tiberius deep learning gene predictor on demand. The server extracts the sequence, runs Tiberius (as a bare process or inside a Singularity container), parses the output, and imports the predicted gene structures as standard annotation features -- immediately editable with full undo/redo support. This avoids the traditional workflow of running genome-wide predictions externally and importing results manually. Planned improvements include an accept/reject preview workflow, RNA-seq evidence input, and Docker container support.

Summary of Changes

Change	Before	After
Assembly/track storage	Single monolithic config document	Individual records with many-to-many relation
Access control	All users see everything	Per-assembly roles with public/private visibilit
Analysis tools	External tools, manual result transfer	Built-in BLAST and Tiberius with job tracking
Database	MongoDB (replica set required)	SQLite, PostgreSQL, or MongoDB via single c

From MongoDB to Relational Databases

Apollo3 migrated from MongoDB to MikroORM, supporting SQLite, PostgreSQL, and MongoDB from a single codebase. The migration is complete. A migration script exists for existing MongoDB deployments.

At a Glance

Data model and editing

Area	MongoDB (before)	Relational (after)
Editing a single feature	Load entire gene doc, modify, write back	Update one row
Two users editing same gene	Second save overwrites first user's changes	No conflict -- separate rows
Data integrity	Application-enforced	Database-enforced (FKs, cascades, tra
Feature lookup by ID	Scan `allIds` arrays across all genes	Direct primary key lookup
Document/row size limits	16MB per gene document	None

Deployment and operations

Area	MongoDB (before)	Relational (after)
Desktop deployment	Not possible (MongoDB requires a server)	Fully self-contained with SQ
Server deployment	2 MongoDB containers in replica set, 4 volumes, init scripts	1 PostgreSQL container, or
Developer setup	Install + configure MongoDB replica set	`pnpm install && pnpm st`
CI setup	MongoDB service container + replica set init	Nothing needed (SQLite)
Min hosting cost	~$50-60/mo (MongoDB Atlas)	0(*SQLite*)*or* 5-15/mo (Post
Backups	`mongodump`	Copy one file (SQLite) or `pg`

Code and maintenance

Area	MongoDB (before)
Server code complexity	Each operation navigated nested trees; many were 30+ lines
Schema definitions	Mongoose schemas + separate `apollo-schemas` package
Dependencies	`mongoose`, `@nestjs/mongoose`, `connect-mongodb-session`, `mongoose-id-validator`,
Real-time collaboration	Unchanged -- runs through WebSockets, independent of DB
Undo/redo	Unchanged -- pure TypeScript, independent of DB

Why We Migrated

1. **Concurrent editing was broken by design.** MongoDB stored an entire gene as one nested document. Two annotators editing different exons both loaded/saved the full document -- second save silently overwrote the first. With flat rows, each feature is its own record.
2. **Every edit touched more data than necessary.** Changing one exon by 1bp loaded and rewrote the entire gene. With flat rows, it's a single-row update.
3. **The `allIds` bookkeeping was fragile.** Every gene carried a manually-maintained list of all descendant IDs. With flat rows, every feature has its own primary key.
4. **16MB document size limit.** Highly spliced genes could hit MongoDB's per-document ceiling. Flat rows have no per-record limit.
5. **Deployment was unnecessarily complex.** MongoDB required a two-container replica set for change streams that Apollo3 didn't use for collaboration (WebSockets handle that). PostgreSQL needs one container; SQLite needs none.

6. **Desktop deployment was not practical.** MongoDB requires a running server process with no embedded mode suitable for desktop apps. SQLite enables fully self-contained desktop/Electron use.

Why MikroORM

- Multi-database from one codebase (SQLite, PostgreSQL, MongoDB, MySQL, MS SQL)
- TypeScript-first entity definitions with compile-time type safety
- Unit of Work pattern: automatic change tracking, single-transaction flush
- Built-in migration system (like Liquibase/Rails migrations)
- First-class NestJS integration via `@mikro-orm/nestjs`

Tradeoffs

Area	Harder?	Mitigation
Loading a full gene tree	Yes -- requires recursive CTEs vs one document fetch	`findDescendantsOfMany` bat
Merging transcripts	Yes -- more DB round-trips for cross-parent operations	Fixable with `UPDATE ... WH`
Text search	Currently weaker (`LIKE` vs MongoDB text indexes)	Replace with SQLite FTS5 o

Serverless and Scale-to-Zero

The relational model enables deployment patterns that were not practical with MongoDB:

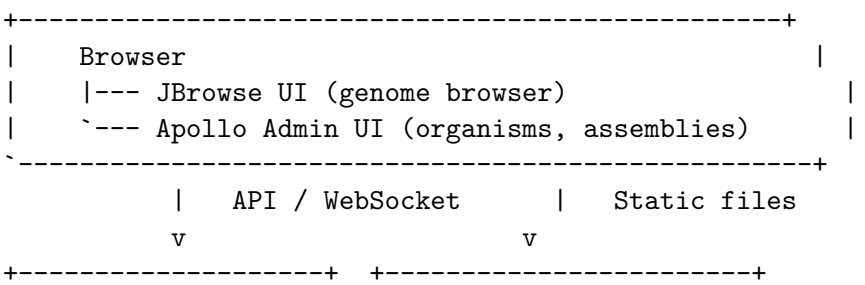
- **Scale-to-zero containers** (Fargate/Cloud Run + Aurora Serverless/Neon): idle cost ~$0-5/mo vs $50-60/mo MongoDB floor
- **Single-file SQLite** on a $3-6/mo nano instance: full Apollo3 in one process
- **Lambda** (read-only): viable for serving annotations to public JBrowse; full collaboration blocked by WebSocket requirement (solvable with SSE)

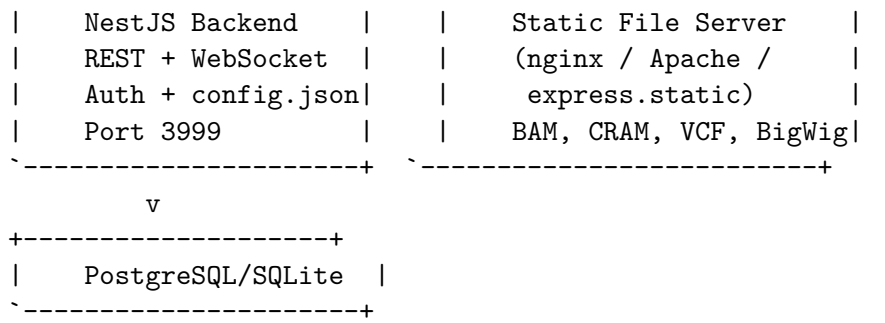
Related Documents

- Benchmark Results -- performance numbers
- Technical Details -- worked examples, schema assessment
- History Tracking -- per-gene history, Apollo2 migration
- Alternatives -- MongoDB fixes, Firestore evaluation

Deployment Guide

Architecture





Deployment Options

Option 1: Single Server (simplest)

NestJS serves everything. Set `JBROWSE_STATIC_DIR` to the JBrowse web directory. Good for development and small teams (< 10 users).

Option 2: nginx + NestJS (recommended for production)

nginx serves static files with zero-copy `sendfile()`. NestJS handles API and WebSocket only. See `deploy/nginx/`.

```
cd deploy/nginx && cp .env.example .env && docker compose up -d
```

Option 3: Apache httpd + NestJS

Same role as nginx. See `.github/workflows/deploy/` for Apache configuration.

Static file performance

Server	Mechanism	Notes
nginx	`sendfile()` -- zero-copy	Multi-process, excellent throughput
Apache	`sendfile()` via `mod_mpm_event`	Multi-threaded, excellent throughput
Node.js	`fs.createReadStream()` with byte offsets	Single-threaded, adequate for small teams

Use nginx/Apache when: BAM/CRAM > 1GB, 10+ concurrent users, or production.

nginx Routing

```
nginx
  |--- /jbrowse/config.json -> proxy to NestJS (dynamic, generated from DB)
  |--- /config.json          -> proxy to NestJS
  |--- /socket.io/           -> proxy to NestJS (WebSocket upgrade)
  |--- /jbrowse/*.bam|cram|bw|html|js -> serve from disk (sendfile)
  |--- /files/<checksum>     -> serve from uploads dir
  `--- everything else       -> proxy to NestJS (API)
```

`JBROWSE_STATIC_DIR` is **not set** on NestJS when using nginx. `config.json` is always proxied (dynamically generated from track/assembly records in DB).

Database

Backend	Use case	Config
SQLite	Dev, desktop, small deployments	Default (no config)
PostgreSQL	Production collaborative	`DB_BACKEND=postgresql DB_CONNECTION_URL=postgresql://`
MongoDB	Legacy (migration from Apollo2)	`DB_BACKEND=mongo DB_CONNECTION_URL=mongodb://...`

Environment Variables

Required

Variable	Description
`URL`	Public URL (e.g. `https://apollo.example.com`)
`NAME`	Instance name shown in UI
`FILE_UPLOAD_FOLDER`	Directory for uploaded files
`PORT`	Server port (default: 3999)

Secrets

Variable	Description
`JWT_SECRET`	JWT signing secret (min 32 chars)
`SESSION_SECRET`	Session cookie secret (min 32 chars)

If not set, the server auto-generates random secrets and persists them to `dist/.secrets/`. Survives restarts. For production, set explicitly or use `_FILE` variants (e.g. `JWT_SECRET_FILE=/run/secrets/jwt`) for Docker secrets.

Optional

Variable	Description
`JBROWSE_STATIC_DIR`	JBrowse static files (single-server only)
`DB_BACKEND`	`sqlite` / `postgresql` / `mongo`
`DB_CONNECTION_URL`	Database connection string
`ALLOW_GUEST_USER`	Allow unauthenticated guest (default: false)
`GUEST_USER_ROLE`	Guest role: `admin` / `user` / `readOnly`
`DEFAULT_NEW_USER_ROLE`	New user role: `admin` / `user` / `readOnly` / `none`
`GOOGLE_CLIENT_ID` / `_SECRET`	Google OAuth
`MICROSOFT_CLIENT_ID` / `_SECRET`	Microsoft OAuth
`ALLOW_ROOT_USER`	Enable root password login
`ROOT_USER_PASSWORD`	Root admin password

First-Time Setup

On first start with no admin, the server prints a one-time setup URL:

`Setup URL: http://localhost:3999/auth/setup?token=<random-token>`

Visit it, then log in (Google, Microsoft, or root). That account becomes admin. Token is single-use. For dev, `GUEST_USER_ROLE=admin` skips this.

Schema Comparison: MongoDB vs Relational

The schema has the same entities. The key change is **how features are stored** and **how relationships are enforced**. For the migration rationale, see mikro-orm-migration-justification.md.

The One Big Change: Feature Storage

MongoDB: nested documents

```
features collection:
+---------------------------------------------+
|   _id: gene_1                               |
|   type: "gene"                              |
|   allIds: [gene_1, mRNA_1, exon_1, exon_2, ...]  |    <- manual bookkeeping
|   children: {                               |
|     mRNA_1: {                               |
|       children: {                           |
|         exon_1: { type: "exon", min: 1000 ... }  |
|         exon_2: { type: "exon", min: 3000 ... }  |
|         CDS_1:  { type: "CDS",  min: 1200 ... }  |
|       }                                     |
|     }                                       |
|   }                                         |
`---------------------------------------------+
```

One record per gene. Every edit loads/saves the whole document. `allIds` must be manually synced. 16MB document size limit.

Relational: flat rows with parent references

```
feature table:
+-------------------------------------------------------------+
|   _id          |   parent       |   type       |   min   |   max   |
|----------------+----------------+--------------+---------+---------|
|   gene_1       |   NULL         |   gene       |   1000  |   5000  |
|   mRNA_1       |   gene_1       |   mRNA       |   1000  |   5000  |
|   exon_1       |   mRNA_1       |   exon       |   1000  |   1500  |
|   exon_2       |   mRNA_1       |   exon       |   3000  |   3500  |
|   CDS_1        |   mRNA_1       |   CDS        |   1200  |   3400  |
`-------------------------------------------------------------+
```

One row per feature. Editing exon_2 touches only that row. No `allIds`. No size limit. `ON DELETE CASCADE` on parent FK handles child cleanup.

Practical comparison

Scenario	MongoDB	Relational
Edit one exon	Load entire gene, modify, write back	Update one row
Two users edit different exons	Second save overwrites first	No conflict -- separate rows
Look up feature by ID	Scan `allIds` arrays	Primary key lookup
Delete a gene	One delete (whole doc)	One delete (CASCADE removes childre
Large gene (thousands of exons)	May hit 16MB limit	No limit
Add/remove child	Update parent's `allIds` + save	Insert/delete one row

Relationship Enforcement

MongoDB relied on application code. The relational schema uses foreign keys with CASCADE delete -- the database enforces relationships automatically.

```
AssemblyEntity
  |---- RefSeqEntity (FK -> assembly, CASCADE)
  |       |---- FeatureEntity (FK -> refSeq, CASCADE; FK -> parent, CASCADE)
  |       |---- RefSeqChunkEntity (FK -> refSeq, CASCADE)
  |       `---- CheckResultEntity (FK -> refSeq, CASCADE)
  `---- ExportEntity (FK -> assembly, CASCADE)
```

Deleting an assembly: one `DELETE` statement. The database removes all refSeqs, features, chunks, check results, and exports automatically.

Unaffected Systems

- Real-time collaboration (WebSockets)
- Undo/redo (TypeScript change classes)
- Change tracking (same audit log structure)
- User management, file handling, quality checks

Performance Fix Discovered During Migration

Quality checks were re-running on every `GET /features/getFeatures` -- thousands of DB queries per pan/zoom. Moved check execution to the mutation pipeline (after edits). Result: **20x speedup** (74s -> 3.65s for 1000 genes). See benchmark-results.md for full numbers.

Technical Details: Relational Migration

Worked examples, tradeoff analysis, and schema assessment. For the rationale, see mikro-orm-migration-justification.md. For deployment options, see deployment.md.

Worked Example: Changing an Exon Boundary

When an annotator moves an exon boundary outward, up to three rows may change: the exon, the transcript (if exon extends beyond it), and the gene.

- **MongoDB**: Scan `allIds` -> load entire gene document (all 50 exons to change

1) -> walk nested structure -> modify one field -> write everything back
- **Relational**: Three targeted single-row updates. Nothing else read or written. No risk of overwriting a concurrent edit to a different exon.

Open question: Both models rely on the client to send correct parent boundary updates. A server-side parent boundary verification after child coordinate changes would make the system defensively correct.

Where Relational Is Harder

Gene tree loading for range queries

The most common read: "get all features overlapping this viewport."

- **MongoDB**: One query returns complete nested gene trees.
- **Relational**: Two steps: (1) find root features by range (indexed, fast), (2) load all descendants via recursive CTE.

`findDescendantsOfMany` already batches all roots into one CTE query. A `root_id` denormalization column has been proposed but introduces a sync burden -- **defer unless profiling proves this is a bottleneck**.

Text search

Current `LIKE`-based search is weaker than MongoDB's text indexes. SQLite FTS5 and PostgreSQL tsvector are both mature built-in replacements requiring no architectural changes.

Tradeoff summary

Operation	Status	Notes
Gene tree loading	Acceptable	Recursive CTEs batch efficiently
Gene/assembly deletion	**Fixed**	`ON DELETE CASCADE` on all FKs
Bulk queries (search, export)	**Fixed**	Batched `IN` filters
N+1 query patterns	**Fixed**	Level-batched BFS, in-memory parent maps
Feature counting	**Fixed**	`em.count()` instead of load-all
Full-text search	Fixable	FTS5 / tsvector
Single-feature edits	Faster	One row vs full document
Concurrent edits	Safer	Separate rows, no contention
Large imports	Better	Streaming, transactions, no size limit

Schema Assessment

What is solid

- One row per feature with parent FK, coordinate columns, composite index on `(refSeq, min, max)`
- 13 entities in ~330 lines, each mapping to a table with typed columns
- Clean separation: entities define structure, repositories handle queries, change classes contain business logic

Implemented improvements

1. **CASCADE deletes** on all FKs -- assembly deletion is a single `deleteById`
2. **Index on `FeatureEntity.parent`** -- tree traversal uses indexed queries
3. **Index on `RefSeqEntity.assembly`** -- assembly lookups indexed
4. **Index on `CheckResultEntity.name`** -- check filtering indexed
5. **Batched descendant queries** -- BFS with `IN` filters (D queries per tree depth, typically 3-4, vs N queries per node)
6. **In-memory parent map for search** -- eliminates per-match DB queries

Remaining improvements

- **(Deferred) `root_id` column** -- single-query tree loading, but requires sync on reparent. Only if profiling justifies it.
- **Normalize `CheckResultEntity.ids`** -- junction table for indexed per-feature lookups instead of JSON `LIKE` filtering.

Schema relationships

```
AssemblyEntity
  `--- RefSeqEntity (FK: assembly)
       |--- FeatureEntity (FK: refSeq; FK: parent -> self)
       |--- RefSeqChunkEntity (FK: refSeq)
       `--- CheckResultEntity (FK: refSeq)

ChangeEntity (FK: reverts -> self, for undo chain)
ExportEntity (FK: assembly)
UserEntity, FileEntity, CheckEntity, JBrowseConfigEntity, CounterEntity (standalone)
```

All FKs have CASCADE delete.

Schema Migrations

MongoDB had no formal migration system. MikroORM provides timestamped migration files (like Liquibase/Rails) that are committed to the repo, run in order on deploy, and can be rolled back.

Currently using `SchemaGenerator.updateSchema()` at startup (appropriate for development). Production should transition to explicit committed migration files for auditability.

Multi-Database Repository Factory

`DB_BACKEND`	Feature Repository	Tree Strategy
`sqlite` (default)	`MikroOrmFeatureRepository`	Recursive CTEs
`postgresql`	`MikroOrmFeatureRepository`	Recursive CTEs
`mongo`	`MongoFeatureRepository`	Iterative BFS via generic EntityManager

All other repositories use `MikroOrm*Repository` implementations with the generic `EntityManager` API (works with any driver).

Alternatives to the MikroORM Migration

Evaluation of other paths. For the chosen approach, see mikro-orm-migration-justification.md.

Option 1: Stay on MongoDB With Targeted Fixes

Phase 1 -- Eliminate `allIds` bottleneck (1-2 weeks)

Replace the `allIds` array scan with a secondary index or lookup collection mapping feature IDs to their parent document.

Phase 2 -- Add concurrency protection (2-3 weeks)

Optimistic locking (version field) or pessimistic locking to prevent one annotator's save from overwriting another's. Doesn't eliminate the root issue (full document load/save) but prevents silent data loss.

Phase 3 -- Partial document flattening (4-6 weeks)

Split at the transcript level: each transcript becomes its own document (exons still nested inside). Reduces blast radius -- editing exon_1 no longer loads unrelated mRNA_2.

Remaining problems: Two annotators editing different exons in the *same* transcript still conflict. No Electron/desktop support. Replica set still required. `allIds` still needed at the transcript level.

Assessment

Comparable effort to the relational migration. Addresses some concurrency issues but not fully (same-transcript conflicts remain). Does not address desktop deployment or simplify the operational setup.

Option 2: Firestore / Firebase

Why tempting: Built-in auth (Google, Microsoft, email), serverless Cloud Functions, real-time sync, zero infrastructure. Would replace ~15 files of Passport/JWT/session code.

Why problematic for Apollo3:

Issue	Impact
No MikroORM driver	Would need to replace the ORM entirely, losing SQLite/PostgreSQL portability
Vendor lock-in	Proprietary to Google Cloud; no standard SQL; pricing subject to change
No offline/Electron	Requires network to Google servers -- same hard constraint as MongoDB
No WebSocket support	Cloud Functions don't support persistent connections; would need to rearchitect collabo
Still document-based	No FKs, no cascades, no joins, no standard query language

Viable hybrid: Use Firebase Authentication (standalone) while keeping MikroORM for data. This captures the auth simplification without database lock-in. Compatible with the relational migration as an independent improvement.

Option 3: Parallel Backend Implementations

The repository interface pattern technically allows multiple backend implementations. In practice, maintaining two complete repository sets, test suites, and deployment configs roughly doubles maintenance burden. The MikroORM migration period (when both MongoDB and relational paths coexisted) confirmed this cost.

Only worthwhile if two fundamentally different deployment targets are needed. The relational model already covers desktop (SQLite) through production server (PostgreSQL).

Recommendation

The relational migration addresses desktop deployment, operational complexity, data model challenges, and hosting cost in one coherent change. If not preferred, a pragmatic fallback:

1. Targeted MongoDB fixes (Phases 1-2) for near-term stability
2. Firebase Auth as standalone service for auth simplification
3. Revisit relational migration when desktop/Electron becomes a priority

Performance Optimization Report

7-20x speedups across all major operations on a 5,000-feature synthetic dataset.

Results

Operation	Before	After	Speedup
Assembly import (5000 features)	60s	8.06s	**7.5x**
Feature get (all)	74s	3.65s	**20x**
Feature search	4.5s	3.41s	1.3x
GFF3 export	6.3s	3.86s	1.6x
Assembly delete	5.0s	3.46s	1.4x

Remaining ~3-4s baseline is CLI startup + HTTP overhead, not DB operations.

What Changed

1. SQLite WAL mode + synchronous tuning

`PRAGMA journal_mode = WAL` + `PRAGMA synchronous = NORMAL` at startup. Concurrent reads during writes, reduced fsync for batch inserts.

2. Raw SQL for read-only queries

Replaced `em.find()` with `em.getConnection().execute()` for all read-only feature queries. ORM hydration (proxy creation, identity map, change tracking) was pure overhead since results were immediately converted to plain objects.

3. Recursive CTEs for tree operations

Replaced iterative BFS (N queries per depth level per root) with single recursive CTE queries for `findDescendantsOfMany`, `deleteDescendants`, and `findRootParent`. For 5000 features across ~1000 gene trees: hundreds of queries -> 1.

4. Moved check recalculation from GET to mutation pipeline

The single largest performance issue -- responsible for the 20x speedup.

`GET /features/getFeatures` was re-running all quality checks on every root feature in the response. For 1000 genes, each pan/zoom triggered: 1000x findById + 1000x findDescendants + 1000x assembleFeatureTrees + check config lookups + delete/rerun/save checks. All redundant -- results were already persisted from the last edit.

Fix: checks run after mutations only. GET returns pre-computed results.

Benchmark Environment

```
Dataset: Synthetic (1000 genes, ~5000 features), 3 iterations
Node.js v24.13.0, linux x64, 2026-03-14
```

Reproduction

```
cd packages/apollo-cli && pnpm tsx src/test/benchmark.ts --synthetic
```

Bug Fixes, Simplifications & Code Quality

Bugs Fixed

1. updateChecks iterating check results as features

`assemblies.service.ts` -- `findByRange()` returned `[features, checkResults]`. The code iterated both arrays, passing check result IDs to `checkFeature()`. **Fix**: Split into `findFeaturesByRange` (features only). Check results fetched separately via `GET /checks/range`.

2. RefSeqsService.remove() deleted wrong scope (dead code)

Looked up one refSeq by ID, then called `deleteByAssembly()` which deletes ALL refSeqs for the assembly. **Fix**: Simplified to accept `assemblyId` directly.

3. User location endpoint encoding bug

Frontend sent `URLSearchParams(JSON.stringify(locations))` -- garbled data, 500 errors on every location update. **Fix**: Proper JSON with `Content-Type: application/json`. Also simplified from array of all visible regions to single primary region (users view one region at a time).

Architectural Simplifications

Developer setup

From 4 parallel processes (shared watch + NestJS + plugin dev server + sibling jbrowse-components clone via justfile) to single NestJS server on port 3999. Removed `npm-run-all`, `concurrently`, `serve`, justfile. Setup is now `pnpm install && pnpm start`.

WebSocket channels

From per-refSeq channels (`${assemblyId}-${refSeqName}`) to single `COMMON` channel. Removed 12 lines of DB queries per change (feature->refSeq->name lookups), `ensureAssemblySocket()` (25 lines), `haveDataForChange()` (12 lines). Safe because annotation edit volume is human-speed. Channel name constants extracted to shared `Messages.ts`.

InternetAccount removal

Removed `ApolloInternetAccount` JBrowse abstraction (~500 lines, 6 files). Cookie auth made it redundant -- `credentials: 'same-origin'` handles everything. WebSocket and change tracking moved to session model. `baseURL`, `role`, `userId` now in `ApolloPlugin` config instead of JWT decode. See internet-account-removal.md for details.

Code Simplification

Change	Before	After
API separation	`getFeatures` returned `[features, checkResults]` tuple	Separate endpoints, pa
Export service	111-line monolith	Focused helpers (`write`
GFF3 export	Duplicate hierarchy assembly (~40 lines)	Shared `assembleFeatu`
ServerDataStore	Anonymous function wrappers around `filesService` methods	Direct `filesService` r
findByFeatureIds	Set-based dedup after `SELECT DISTINCT`	DB handles it
RefSeqsService.update	12-line manual property mapping	Spread operator (4 line
Dev Container	MongoDB extension + `mongosh` + port 27017	PostgreSQL + port 543

Authentication & Security Audit

Two rounds of audits covering authentication, controllers, data handling, and injection surfaces. All critical/high issues fixed.

Issues Fixed

#	Severity	Issue
1	CRITICAL	**Open redirect** -- OAuth callback accepted arbitrary `redirect_uri`, allowing token theft via
2	CRITICAL	**Missing Secure flag** -- JWT cookie sent over plain HTTP
3	HIGH	**WebSocket CORS wildcard** -- `cors: { origin: '*' }` on WebSocket gateway
4	HIGH	**JWT logged in plaintext** -- full token + user object at DEBUG level
5	BUG	**OAuth client ID file-read** -- file contents overwritten by file path (`microsoftClientID = c`
6	MEDIUM	**Session cookies lacked security options** -- no `httpOnly`, `secure`, `sameSite`, `maxAge`

#	Severity	Issue
7	MEDIUM	**No minimum secret length** -- single-char secrets accepted

Verified Secure

- All 14 controllers have class-level auth decorators; default is `Role.Admin`
- SQL queries use MikroORM parameterized queries (no injection risk)
- File uploads store by checksum, not user-provided filename (no path traversal)
- Root password comparison uses plaintext `===` against env var (intentional -- hashing env vars provides no security since attacker with env access already has the password)

Accepted Risks

- Token in OAuth redirect URL: mitigated by origin validation + short-lived popup
- No CSRF middleware: mitigated by `SameSite=lax`
- No refresh token: 24-hour JWT expiry is acceptable
- No token revocation on logout: standard for stateless JWT

CheckResult Data Model Simplification

Change

Replaced `ids: string[]` JSON array on `CheckResultEntity` with a single indexed `featureId: string` column.

Why

Both existing check implementations produce exactly one feature ID per result. The array was designed for a multi-feature check scenario that has not been needed. If that scenario arises in the future, a junction table would be a better fit than a JSON array.

The `LIKE`-based query on the JSON blob had a false-positive matching bug (`"feat-1"` could match `"feat-123"`), required a two-stage query+filter pattern, and prevented indexing.

Impact

Aspect	Before	After
Query	`LIKE '%"id"%'` + in-memory filter (full table scan)	Indexed `WHERE featureId = ?` (O(log n))
Entity	`ids: p.json<string[]>()`	`featureId: p.string()` + index
MST	`types.array(types.safeReference(...))`	`types.safeReference(...)`
Repository	121 lines, 17-line two-stage delete	96 lines, 3-line direct delete

Also Done: Repository Instance Caching

`DatabaseService` previously created new repository instances on every getter call. Now created once in constructor and reused -- eliminates ~12 object allocations per HTTP request. Safe because `RequestContext` middleware provides per-request EM isolation via `AsyncLocalStorage`.

InternetAccount Removal

Change

Removed the `ApolloInternetAccount` JBrowse plugin abstraction (~500 lines, 6 files). Replaced with direct cookie-based auth and session-level connection management.

Why

After migrating to cookie-based auth (HTTP-only cookies), the InternetAccount was redundant:

- `CollaborationServerDriver.fetch()` already used `credentials: 'same-origin'`
- Cookie auth is handled transparently by the browser
- Multi-account selection UI added complexity for a feature no deployment uses (Apollo3 is single-server)

What Moved Where

Concern	Before	After
WebSocket management	`ApolloInternetAccount/model.ts`	Session model (`session.ts`)
Change sequence tracking	`ApolloInternetAccount/model.ts`	Session model
`baseURL`, `role`, `userId`	JWT token decode + `internetAccounts` config	`ApolloPlugin` config in `config.j`
API calls	`internetAccount.getFetcher()`	`apolloFetch()` with `credentials`
Multi-account UI	~200 lines across 6 components	Deleted

Login Flow (Current)

1. Plugin reads `baseURL` from `ApolloPlugin` configuration
2. Session fetches `${baseURL}/jbrowse/config.json` with cookie credentials
3. Server returns config with `role`, `userId`, and assemblies (if authenticated)
4. Session initializes WebSocket and admin menus based on role

Impact

Metric	Before	After
InternetAccount files	6	0
Auth mechanisms	Cookie + JWT + Authorization header	Cookie only
Plugin bundle	1.57 MB	1.56 MB

Technical Notes

Deep technical analysis of the flat-row data model, migration cleanup, and remaining improvement opportunities.

Flat Rows vs Nested Documents: Operation-by-Operation

Single-feature edits (most common)

All single-feature operations (change coordinates, type, strand, attributes) follow the same pattern:

- **MongoDB**: Load entire gene doc -> walk tree -> modify -> save whole doc (~70 lines)
- **Flat rows**: Look up one row -> update -> done (2 queries, ~15 lines)

Flat rows produce shorter code, fewer database reads/writes, and less risk of unrelated data being touched. The tradeoff is that features are now on separate rows, so operations that need the full gene tree (like validation checks) require an extra assembly step -- a recursive query to collect descendants. For single edits this cost doesn't apply.

Adding features

Operation	MongoDB	Flat rows
New top-level gene	Create one nested doc	Flatten sn
Add exon to existing mRNA	Load gene, insert child, update `allIds`, save whole doc	Insert one
GFF3 import	One doc at a time, no transactions (16MB limit), OOM on large files	Flatten to

Deleting features

Operation	MongoDB	Flat rows
Delete top-level gene	Delete one document	`ON DELETE CASCADE` han
Delete exon from mRNA	Load gene, find exon in tree, remove, update `allIds`, save	Delete one row

Structural edits

Operation	MongoDB	Flat rows
Split exon	Load gene, create two children, update `allIds`, save	Insert two rows, delete old row
Merge exons	Load gene, merge in memory, save	Update first exon bounds, delete secon
Merge transcripts	All in-memory on one document, single save	Query children separately, reparent, d

Merging transcripts is the clearest case where nested documents had an advantage -- all children were already in memory. With flat rows, this requires separate queries and reparenting, though batching can reduce the overhead.

Undo operations

Delete what forward created, re-insert what forward deleted. No tree navigation or `allIds` bookkeeping. Simpler in the flat model.

Read operations

Operation	MongoDB	Flat rows
Features in coordinate range	Returns full nested trees (loads more than needed)	Returns exactly matching
Find feature by ID	`findOne({allIds: id})` + tree walk	Primary key lookup
Find all CDS on a chromosome	Load all genes, walk all trees	`WHERE type='CDS' AND r`
Count features by type	Load all, count in app code	`GROUP BY type`
Export to GFF3	Data already nested	Rows map to GFF3 lines
Run validation checks	Data already nested	Fetch descendants + asser
Text search	`$text` index (slow writes)	Currently `LIKE` only. Fixa

For targeted queries (by ID, by type, by range), the relational model benefits from standard database indexing. For operations that need the full gene tree (validation, client display), there is an extra step to collect and reassemble descendants. MongoDB returned these pre-assembled, which was convenient. The cost of reassembly is modest (one recursive CTE query + O(n) in-memory pass), but it is a real tradeoff.

Operations MongoDB couldn't do well

- **Reparent a feature**: Flat rows: `UPDATE SET parent = :new WHERE _id = :id`. MongoDB: load both source/destination gene docs, move between nested Maps, update both `allIds`, save both docs.
- **Query across hierarchy**: "genes with exons < 50bp?" -- a WHERE clause with flat rows. MongoDB: load all gene docs, walk every tree.
- **Stream imports**: Flat rows stream to DB with bounded memory. MongoDB required building nested trees in memory first; OOM on large files.

Summary

Simple edits (the majority of annotation work) are shorter code, faster, and touch less data with flat rows. Complex structural edits like transcript merges require more database round-trips, though this is addressable with batch queries. Read operations gain precise indexed queries at the cost of a tree-assembly step when the full hierarchy is needed. The nested document model avoided that assembly step, but at the cost of loading and rewriting entire genes for every operation -- including simple ones.

Migration Cleanup

Unified execution path

Removed dual `executeOnServer()` / `executeOnServerV2()` implementations from all 23 Change classes and 2 Operation classes. Removed `ServerDataStoreV2`. Single code path.

Eliminated FeatureChange helpers

Five tree navigation methods (`getFeatureFromId`, `getChildFeatureIds`, `generateNewIds`, `addChild`, `findAndDeleteChildFeature`) were specific to the nested document model and no longer needed after the switch to flat rows. The MongoDB backend retains its own tree traversal via `MongoFeatureRepository`.

Removed dead code

- `backendPostValidate()` -- never called from active code path. Entire validation layer (`ParentChildValidation`, `ValidationSet`) removed.
- `LocalGFF3DataStore` / `executeOnLocalGFF3` -- never implemented (all threw "not implemented"). Removed from all Operations and Changes.
- `allIds` on `AddFeatureChangeDetails` -- carried forward as dead weight in serialized change format. Removed from interface and all client code.
- `@apollo-annotation/schemas` dependency -- Mongoose schema types removed from `apollo-common` and `apollo-shared`.

Potential Optimizations

R-tree spatial indexes

Current B-tree on `(refSeq, min, max)` narrows on one bound at a time. R-tree indexes (SQLite native, PostgreSQL GiST) prune on both bounds simultaneously -- useful for dense feature regions.

Bulk import via COPY

PostgreSQL's `COPY FROM` inserts millions of rows/second from flat files. A GFF3 -> CSV -> COPY pipeline would be faster than ORM-level insertion.

Explicit migrations

Currently using `SchemaGenerator.updateSchema()` at startup. Transitioning to committed migration files would provide auditable, reversible schema evolution -- restoring the discipline Apollo2 had with Liquibase.

NestJS Code Issues Found

Issue	Location	Fix
No DTO validation (invalid data reaches service layer)	7 DTO files	Add
ChangesService has too many responsibilities	`changes.service.ts`	Extra
Duplicated ServerDataStore factory	`changes.service.ts`, `operations.service.ts`	Extra
Silent auth failures (generic 403)	`validation.guards.ts`	Throw
Duplicate OAuth guards	`google.guard.ts`, `microsoft.guard.ts`	Gene
Inefficient admin check on login	`authentication.service.ts`	`count`

Security issues (OAuth file-read bug, open redirect, etc.) are tracked in the *Authentication & Security Audit* section.

Per-Gene History Tracking and Apollo 2 Migration

The Problem

Apollo3's change log is a **global stream** -- every edit is recorded, but there's no way to query "all edits to gene X" without scanning the entire change table. Apollo2 had per-gene history via Grails audit logging.

The `changedIds` field stores affected feature IDs as a JSON blob. JSON arrays can't be efficiently indexed in SQLite, and PostgreSQL JSONB GIN indexes require database-specific syntax that breaks cross-DB portability.

The Fix: `gene_id` Column

Add an indexed `gene_id` column to `ChangeEntity`. Each feature change records its top-level gene's ID (resolved via `findRootParent`).

```
SELECT * FROM change WHERE gene_id = ? ORDER BY sequence DESC
```

Standard indexed lookup, works identically in SQLite and PostgreSQL, O(log n).

Comparison

Query	Apollo2	Apollo3 (current)	Apollo3 (proposed)
History of gene X	Direct audit table query	Full table scan of JSON	Indexed `WHERE gene_id = ?`
Changes by user Y	Scan	Indexed	Same
Changes since time T	Scan	Indexed by `sequence`	Same
Undo support	Limited (view-only)	Full (`getInverse()`)	Same

Implementation

Phase 1 -- Backend:

1. Add nullable indexed `gene_id` column to `ChangeEntity`
2. Resolve top-level gene in `ChangesService.create()` via `findRootParent`
3. Add `GET /changes?geneId=<id>` endpoint
4. Backfill migration for existing change rows

Phase 2 -- Frontend:

1. "View History" in gene context menu
2. Timeline display: user, timestamp, human-readable change summary
3. "Undo this change" button (uses existing `getInverse()`)

Apollo 2 Data Migration

The problem with GFF3-only migration

Exporting GFF3 from Apollo2 and importing into Apollo3 preserves annotations but **loses all edit history** -- who created/modified each gene, when, and what previous states were.

Direct history migration

Map Apollo2 audit records to Apollo3 `ChangeEntity` rows:

Apollo2 field	Apollo3 mapping
Audit operation (INSERT/UPDATE/DELETE)	`typeName` (AddFeature/*/DeleteFeature)
User	`user`

Apollo2 field	Apollo3 mapping
Timestamp	`createdAt` (preserved)
Property changes	`changes` (JSON)
Feature -> organism/sequence	`assembly`
Feature -> top-level gene	`gene_id`

Imported records use a `Apollo2Import:` prefix on `typeName` -- viewable but **not undoable** (no `getInverse()` implementation).

Combined workflow: Apollo 2 -> Apollo 3

1. Export annotations from Apollo2 as GFF3
2. Run history migration script against Apollo2 PostgreSQL
3. Import GFF3 into Apollo3
4. Verify per-gene history shows unified timeline